

# Performance Management for Cloud Services: Implementation and Evaluation of Schedulers for OpenStack

HANS LINDGREN



**KTH Electrical Engineering**

Master's Degree Project  
Stockholm, Sweden July 2013

XR-EE-LCN 2013:012



KTH Electrical Engineering

# Performance Management for Cloud Services: Implementation and Evaluation of Schedulers for OpenStack

HANS LINDGREN

Supervisors:

Fetahi Wuhib, KTH

Hareesh Puthalath, Ericsson Research

Examiner:

Prof. Rolf Stadler

Stockholm July 2013

---

Communication Networks  
School of Electrical Engineering  
Kungliga Tekniska Hgskolan

XR-EE-LCN 2013:012

## Abstract

To achieve the best performance out of an IaaS cloud, the resource management layer must be able to distribute the workloads it is tasked with optimally on the underlying infrastructure. A utilization-based scheduler can take advantage of the fact that allocated resources and actual resource usage often differ to make better-informed decisions of where to place future requests.

This thesis presents the design, implementation and evaluation of an initial placement controller that uses host utilization data as one of its inputs to help place virtual machines according to one of a number of supported management objectives. The implementation, which builds on top of the OpenStack cloud platform, deals with two different objectives, namely, balanced load and energy efficiency. The thesis also discusses additional objectives and how they can be supported.

A testbed and demonstration platform consisting of the aforementioned controller, a synthetic load generator and a monitoring system are built and used during evaluation of the system. Results indicate that the scheduler performs equally well for both objectives using synthetically generated request patterns of both interactive and batch type workloads.

A discussion of current limitations of the scheduler and ways to overcome those conclude the thesis. Among the things discussed are how the rate at which host utilization data is collected limits the performance of the scheduler and under which circumstances dynamic placement of virtual machines must be used to complement utilization-based scheduling to avoid the risk of overloading the cloud.

## Sammanfattning

För att erhålla maximal prestanda ur ett IaaS moln måste dess resurshanteringssystem kunna schemalägga resursutnyttjandet av den underliggande infrastrukturen på ett optimalt sätt. En nyttjandebaserad schemaläggare kan dra nytta av det faktum att allokerade resurser och faktiskt använda resurser ofta skiljer sig åt, för att på så sätt kunna fatta mer välgrundade beslut om var framtida förfrågningar skall placeras.

Detta examensarbete presenterar såväl utformning, implementation samt utvärdering av en kontrollenhet för initial placering som till sin hjälp använder information om värdutnyttjande som indata för placering av virtuella maskiner enligt ett av ett antal stödda förvaltningssyften. Implementationen, som baseras på molnplattformen OpenStack, behandlar två sådana syften, balanserad last och energieffektivitet. I tillägg diskuteras krav för att stödja ytterligare syften.

En testmiljö och demonstrationsplattform, bestående av ovan nämnda kontrollenhet, en syntetisk lastgenererare samt en övervakningsplattform byggs upp och används vid utvärdering av systemet. Resultat av utvärderingen påvisar att schemaläggaren presterar likvärdigt för båda syften vid användande av last bestående av såväl interaktiva som applikationer av batch-typ.

En diskussion om nuvarande begränsningar hos schemaläggaren och hur dessa kan överbryggas sammanfattar arbetet. Bland det som tas upp diskuteras bl.a. hur hastigheten vid vilken värdutnyttjande data samlas in påverkar prestandan hos schemaläggaren samt under vilka förhållanden dynamisk placering av virtuella maskiner bör användas som komplement till nyttjandebaserad schemaläggning för att undvika risk för överbelastning i molnet.

## Acknowledgements

This thesis was carried out as part of the joint collaboration between KTH, Laboratory of Communication Networks and Ericsson Research around their involvement in the EU SAIL project. I would like to take the opportunity to thank all those being directly or indirectly involved in making this project happen.

First of all, I would like to thank my supervisors, Fetahi Wuhib, KTH, and Hareesh Puthalath, Ericsson Research for their guidance and inspiration throughout this project. I would also like to thank Prof. Rolf Stadler for his support and for giving me the opportunity to work on this thesis. Furthermore I would like to thank Enrique Fernández, Bob Melander and the entire crew at Ericsson Research for their welcoming attitude and willingness to help, which made my work there a joyful experience. Finally I would like to express a special thanks to Rerngvit Yanggratoke, KTH for his assistance in making necessary preparations needed to finish this thesis.

Last but not least, I would like to express gratitude to my family, girlfriend and close friends for their unyielding support and tolerance throughout this thesis and the years of hard work precluding it.

## Contents

|                                                                |     |
|----------------------------------------------------------------|-----|
| List of Figures .....                                          | vii |
| List of Tables.....                                            | vii |
| 1 Introduction .....                                           | 1   |
| 1.1 Motivation.....                                            | 1   |
| 1.2 Problem.....                                               | 2   |
| 1.3 Thesis Objectives.....                                     | 2   |
| 1.4 Thesis Plan.....                                           | 3   |
| 1.5 Thesis Contribution .....                                  | 3   |
| 1.6 Thesis Outline.....                                        | 4   |
| 2 Background .....                                             | 5   |
| 2.1 Cloud Computing.....                                       | 5   |
| 2.1.1 Cloud Service Classifications .....                      | 6   |
| 2.1.2 Cloud Infrastructure Software .....                      | 7   |
| 2.2 OpenStack .....                                            | 8   |
| 2.2.1 History .....                                            | 8   |
| 2.2.2 OpenStack Compute .....                                  | 8   |
| 2.2.3 Compute Scheduler.....                                   | 9   |
| 2.3 Cloud Performance Management.....                          | 10  |
| 2.3.1 Management Objectives .....                              | 10  |
| 2.3.2 Constraints .....                                        | 10  |
| 2.3.3 Resource Management.....                                 | 10  |
| 3 Related Work .....                                           | 11  |
| 4 Initial Placement Controller .....                           | 13  |
| 4.1 Controller Architecture .....                              | 13  |
| 4.2 Controller Input.....                                      | 14  |
| 4.2.1 Placement Policies .....                                 | 14  |
| 4.2.2 Request Details .....                                    | 14  |
| 4.2.3 Cloud State .....                                        | 14  |
| 4.3 Management Objectives .....                                | 15  |
| 4.3.1 Reservation-Based vs. Utilization-Based Scheduling ..... | 15  |

|       |                                                         |    |
|-------|---------------------------------------------------------|----|
| 4.3.2 | Online Algorithms .....                                 | 15 |
| 4.3.3 | Balanced Load Objective .....                           | 16 |
| 4.3.4 | Energy Efficiency Objective .....                       | 17 |
| 4.3.5 | Minimize Network Bandwidth Objective .....              | 18 |
| 4.3.6 | Fair Allocation Objective .....                         | 18 |
| 4.3.7 | Service Differentiation Objective .....                 | 19 |
| 4.3.8 | Constraints .....                                       | 19 |
| 5     | Implementation in OpenStack .....                       | 21 |
| 5.1   | OpenStack Compute Scheduler .....                       | 21 |
| 5.1.1 | Brief History of the Compute Scheduler .....            | 21 |
| 5.1.2 | Overview of the nova-scheduler service.....             | 22 |
| 5.1.3 | Extended Version of Least-Cost Scheduler .....          | 23 |
| 5.2   | Utilization Reporting .....                             | 24 |
| 5.2.1 | Database Extensions .....                               | 25 |
| 5.2.2 | Driver Reporting.....                                   | 25 |
| 5.3   | Performance Monitoring .....                            | 26 |
| 5.3.1 | Zabbix Monitoring Package.....                          | 26 |
| 5.3.2 | Custom-Built User Interface.....                        | 26 |
| 5.3.3 | Zabbix User-Defined Monitoring Items.....               | 27 |
| 6     | Scheduler Evaluation.....                               | 28 |
| 6.1   | Testbed Details.....                                    | 28 |
| 6.1.1 | Cloud Servers .....                                     | 28 |
| 6.1.2 | OpenStack Cloud Setup.....                              | 28 |
| 6.1.3 | VM Image Configurations .....                           | 29 |
| 6.1.4 | Instance Request Generation.....                        | 29 |
| 6.1.5 | Zabbix Monitoring Setup.....                            | 30 |
| 6.2   | Cloud Performance Metrics .....                         | 30 |
| 6.2.1 | Individually Collected Host Metrics.....                | 30 |
| 6.2.2 | Aggregated Metrics for Balanced Load Objective.....     | 30 |
| 6.2.3 | Aggregated Metrics for Energy Efficiency Objective..... | 31 |
| 6.3   | Experiments .....                                       | 32 |

|       |                                                 |    |
|-------|-------------------------------------------------|----|
| 6.3.1 | Experiment 1: Balanced Load Objective .....     | 33 |
| 6.3.2 | Experiment 2: Energy Efficiency Objective ..... | 35 |
| 7     | Results Discussion and Future Work.....         | 36 |
| 7.1   | Conclusions .....                               | 36 |
| 7.2   | Experiences .....                               | 38 |
| 7.3   | Future Work .....                               | 38 |
| 8     | Bibliography .....                              | 39 |



## List of Figures

|                                                                               |    |
|-------------------------------------------------------------------------------|----|
| Figure 1: The different layers of an IaaS cloud .....                         | 7  |
| Figure 2: OpenStack Compute architecture overview .....                       | 9  |
| Figure 3: Initial Placement Controller Decision Engine .....                  | 13 |
| Figure 4: Ways to distribute the same load across two servers .....           | 16 |
| Figure 5: Diablo scheduler driver classes .....                               | 23 |
| Figure 6: Request flow of the least-cost scheduler .....                      | 23 |
| Figure 7: Modified database schema of the compute_nodes table .....           | 25 |
| Figure 8: Screenshot of the custom-built monitoring interface .....           | 27 |
| Figure 9: Balanced Load performance for the duration of a test run .....      | 34 |
| Figure 10: Summarized results for Balanced Load objective .....               | 34 |
| Figure 11: Energy Efficiency performance for the duration of a test run ..... | 35 |
| Figure 12: Summarized results for Energy Efficiency objective .....           | 36 |

## List of Tables

|                                                    |    |
|----------------------------------------------------|----|
| Table 1: Images and VM configurations used .....   | 32 |
| Table 2: VM arrival rates and lifetimes used ..... | 33 |

# 1 Introduction

The dream of utility computing with little or no upfront investments, scale to your needs and pay-as-you-go has turned the IT industry's focus towards that of cloud computing. Behind the scenes, the technology that drives the cloud is nothing revolutionary new, but rather an adapted use of existing technology that naturally fits how services nowadays are consumed on the Internet.

Large datacenters located around the world that make use of virtualization techniques to group lots of commodity servers into large resource clusters, are exposed to its users as services through standardized web interfaces. All in all, hiding the complexity of the underlying infrastructure, providing the illusion of an infinite pool of resources made available for on-demand use to instantly serve user needs as they arise.

The most commonly used cloud service; the so called Infrastructure-as-a-Service (IaaS) cloud, being pioneered by Amazon Web Services with their Elastic Compute Cloud (EC2) product [1], give the user access to virtual machines (VMs) with similar characteristics to that of ordinary physical machines. With such VMs, the user can freely choose to install arbitrary software and operating systems that best suits the job being carried out.

## 1.1 Motivation

The IaaS cloud model has two main stakeholders; (1) the service provider who owns and operates the cloud and (2) cloud consumers, which host their applications within VMs in the cloud. While the benefits to the cloud consumer are obvious, the management burden now is something that the service provider must handle. In doing so, the provider faces the challenge of making efficient use of the combined cloud resources to reduce operational costs, while still adhering to customer service level agreements (SLAs). As competition grows, with more providers entering this increasingly popular market, service prices are becoming an important deciding factor for consumers. As a result, making efficient use of the investments in infrastructure becomes even more critical to stay competitive. Despite this, most IaaS resource management systems still relies on overly simple methods to carry out resource management, with resource partitioning being the predominant scheme used to assign resources to customers' virtual machines. While this scheme has the benefit of always guaranteeing SLAs, it potentially wastes a lot of valuable resources as unused resources of underutilized VMs cannot be put to use by other VMs. To overcome some of this, over-allocation, where you simply assign more resources than you have, hoping that not all VMs do fully utilize their resources at the same time, is commonly being used. A more controlled way of doing this than to just keep track of static resource allocations, is to monitor the actual utilization of host resources to be able to make better use of statistical multiplexing to be able to pack VMs more efficiently. If coupled with a dynamic placement strategy

where VM placements continuously adapt to meet dynamic changes in demand, using techniques like VM live-migration, customer SLAs can still mostly be respected.

With the increased focus on environmental problems, green computing and means to reduce power consumption of cloud datacenters has become a top research priority. This is further reinforced by the fact that energy costs, due to recent price increases, have quickly become one of the main operational expenses of running a cloud datacenter. By consolidating VM workloads onto a minimal number of physical machines, a cloud operator can put unused machines to standby and thereby minimize the energy needed to run its service.

## 1.2 Problem

The specific problem being addressed in this thesis is that of initial VM placement in IaaS clouds. One of the fundamental characteristics of cloud computing is the rapid elasticity of the service in which resources can easily and quickly be provisioned and released, resulting in constant fluctuations in resource demands. In an IaaS cloud, this translates to a constant flow of requests for provisioning and termination of VM instances, making it a challenging task to optimally place requests onto the underlying infrastructure. In addition, this has to be done without knowledge of future requests, complicating things further. Depending on the service being offered as well as other factors, the optimal placement might differ due to service provider objectives not always being the same. What is optimal might also change over time as a result of trends in demand changing over time, like in the case of recurring shifts in load between days/nights, weekdays/weekends etc.

## 1.3 Thesis Objectives

The goal of this thesis is to design and implement an initial placement controller that solves different VM placement problems expressed as management objectives. Implementation will target the scheduler component of OpenStack [2]. Furthermore, the ability of the implemented scheduler to achieve its goals under different management objectives are to be demonstrated using a prototype cloud setup, where experimental evaluation will be carried out to understand its performance characteristics under different settings.

The design will build upon the existing architecture of the scheduler component in OpenStack where the scheduler is assumed to have global knowledge of the state of the whole system for making placement decisions in response to the arrival of new VM requests. The problem is essentially treated like an online scheduling problem where requests for single VM instances are handled sequentially, without knowledge of future requests. Only computational resources (CPU, memory) for placing VMs in the cloud are considered at this time, but the design should be extendable enough to be able to handle other resource types in the future.

The method used for evaluation is to perform different experiments using a testbed consisting of the above described prototype complemented by a monitoring system that allows real-time monitoring and collecting required performance data for analysis. Finally, a randomly generated flow of VM requests are used to simulate user activity in the system. Achieved performance is then compared to the optimal case for the management objective at hand.

## 1.4 Thesis Plan

The plan starts with a literature study on background materials for this thesis, covering the area of cloud computing with a focus on IaaS clouds, the OpenStack platform and its architecture, and more specifically details of the scheduler component. Furthermore, research topics like VM placement and cloud performance management are covered. Following this, an OpenStack cluster is installed and used to help understand the details of this platform and in particular how the scheduler component works. Additionally it will be used for development and testing of a new scheduler during the implementation phase. Later on, this setup is also going to serve as the testbed for evaluation purposes. Before the implementation of a new scheduler can start, a design for an initial placement controller and its algorithms must be completed. Implementation uses an incremental development cycle where at first, just a single management objective is implemented on top of the current available cloud state. Later iterations make necessary redesigns and add to the functionality of the scheduler until it has enough functionality for evaluation to start. In parallel with development of the scheduler, the testbed are finalized, so that early testing can begin and so that demonstrations of the work-in-progress can be shown to an outside audience. This includes setting up the required monitoring capabilities and figuring out a way to generate load in the system, which is vital to the evaluation and demonstration of how the scheduler performs.

## 1.5 Thesis Contribution

This thesis contributes towards better performance management of IaaS clouds by providing a generic system for solving initial VM placement in support of different management objectives. The specific contributions are:

- A software implementation of an initial placement controller with support for an open range of management objectives to achieve efficient performance management in an IaaS cloud.
- Definition of and instantiation of two common management objectives; balanced load and energy efficiency, which are also used when evaluating the performance of the system.
- A test framework for evaluating the performance of the system, including configurable load generation and monitoring of key performance metrics related to the two management objectives.

- A web based demonstration platform used to showcase the performance of the system.
- A performance evaluation and analysis that shows the potential of the built system.

## **1.6 Thesis Outline**

The remaining parts of this thesis are organized as follows. Chapter 2 gives background information on cloud computing in general and in-depth information on OpenStack, which is the cloud computing platform targeted for the implementation parts of this thesis. Chapter 3 examines work related to this thesis. Chapter 4 introduces the design of an initial placement controller that is later used during implementation of the thesis work. Chapter 5 presents details on the implementation of the above controller design in OpenStack together with a monitoring solution, later used for evaluation. Chapter 6 presents the evaluation platform and details the experiments that was conducted to evaluate the system. Finally, chapter 7 presents a summary of the thesis results together with conclusions and possible future work related to the outcome of this thesis.

## 2 Background

This chapter provides the necessary background needed to give both context and deeper understanding of the main contents this thesis. Since the target area of this work is cloud computing, a first section details some of the important concepts of this area and also looks at different classifications of cloud services. Since large parts of this thesis targets the OpenStack platform, a follow up section gives an overview of those parts of the platform that are relevant to this thesis, starting with an architectural overview of its main IaaS software component, focusing on details of one of its components, the scheduler as this is where most of the implementation will take place. A third section looks at resource management concepts and introduces management objectives as a mean to control the resource management process.

### 2.1 Cloud Computing

Cloud computing is a broad term that encompasses many different aspects of a paradigm where compute resources are made available for consumption as a service over a networked connection, like the Internet. As such, a cloud service has two main stakeholders; (1) the service provider who owns and operates the cloud and (2) cloud consumers, by some referred to as the cloud users, who targets requests at, and uses the services made available to them by (1). Vaquero et al. propose the following cloud definition in [3]:

*“Clouds are a large pool of easily usable and accessible virtualized resources (such as hardware, development platforms and/or services). These resources can be dynamically reconfigured to adjust to a variable load (scale), allowing also for an optimum resource utilization. This pool of resources is typically exploited by a pay-per-use model in which guarantees are offered by the Infrastructure Provider by means of customized SLAs.”*

Foster et al. [4] propose a similar definition where they also conclude that the driving force behind the cloud paradigm is that of economies of scale. Armbrust et al. [5] chose a slightly more consumer-oriented perspective when they list three key aspects that are new in cloud computing, namely; (1) the illusion of infinite computing resources available on demand, (2) the elimination of an up-front commitment by cloud users, (3) the ability to pay for use of computing resources on a short-term basis as needed.

From these definitions, it is clear that the management burden is now mostly put on the cloud provider who must efficiently operate a potentially very large environment in a highly dynamic setting with constant non-controllable demand changes, while at the same time conform to customer SLAs.

### 2.1.1 Cloud Service Classifications

The National Institute of Standards and Technology (NIST) proposed another popular definition of cloud computing [6] that in addition to the above definitions provides for classifications of different cloud service and deployment models. It has gained wide acceptance and is frequently used to describe different cloud offerings. The three service models being defined here that can be thought of as being layered upon each other are, from top to bottom:

- *“Software-as-a-Service (SaaS). The capability provided to the consumer is to use the provider’s applications running on a cloud infrastructure. The applications are accessible from various client devices through either a thin client interface, such as a web browser (e.g., web-based email), or a program interface. The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, storage, or even individual application capabilities, with the possible exception of limited user-specific application configuration settings.”*
- *“Platform-as-a-Service (PaaS). The capability provided to the consumer is to deploy onto the cloud infrastructure consumer-created or acquired applications created using programming languages, libraries, services, and tools supported by the provider. The consumer does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage, but has control over the deployed applications and possibly configuration settings for the application-hosting environment.”*
- *“Infrastructure-as-a-Service (IaaS). The capability provided to the consumer is to provision processing, storage, networks, and other fundamental computing resources where the consumer is able to deploy and run arbitrary software, which can include operating systems and applications. The consumer does not manage or control the underlying cloud infrastructure but has control over operating systems, storage, and deployed applications; and possibly limited control of select networking components (e.g., host firewalls).”*

Popular public cloud services belonging to the above service models include SaaS services like Salesforce CRM [7], PaaS services like Google App Engine [8] and finally IaaS services like Amazon EC2 [1]. The focus of this thesis is entirely in the last of those three, but can have an impact on all, since the others are layered on top of this one.

When referring to clouds, another common denotation is to label them as being either public or private depending on the deployment model, also proposed by the NIST definition. Public clouds are offered to a broader general audience (like all the examples of the different service models stated above) while private clouds are restricted for use by the internal consumers of a single organization or entity. The NIST definition, in addition to these often referred deployment models, has two additional models known

as community clouds and hybrid clouds, which are merely variants or combinations of the two former ones. From the perspective of this thesis, the deployment model being used is mostly irrelevant as the role of performance management stays the same no matter what the deployment model is.

Throughout this thesis, the use of the term “cloud”, unless otherwise stated should be interpreted as an IaaS cloud in a public setting.

### 2.1.2 Cloud Infrastructure Software

The currently available software for building IaaS clouds ranges from commercial to freely available open source software packages, each with varying support for constructing the kinds of clouds presented in section 2.1.1. Since commercial software both cost money and generally are locked down, prohibiting making any code changes, they do not suit the purposes of project like this and are therefore considered out of scope. The focus instead is put on open source packages, specifically OpenStack, which is the targeted platform of this thesis. Besides free access to all source code, which enables further development of the code, open source communities generally promote participation by making the source code easily obtainable through a version control system and by providing developer documentation to help newcomers getting started.

The IaaS architecture consists of the different layers shown in Figure 1. The API layer on the very top, which provides the interface the cloud consumers use to interact with the cloud. Beneath it, the management layer that is responsible for orchestrating and to provision the resources of the cloud in response to API requests. Popular public IaaS cloud services like Amazon EC2 [1], Google Compute Engine [9] and Windows Azure [10] use proprietary software for their management, while vendors like Rackspace [11] and HP [12] base their cloud offerings on open source software such as OpenStack.

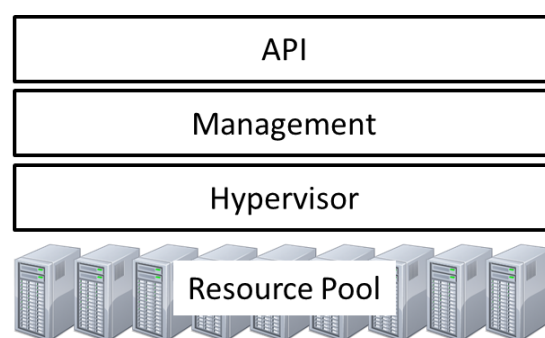


Figure 1: The different layers of an IaaS cloud

Cloud computing and IaaS in particular has adopted hardware virtualization as a key enabler for efficient resource usage. The hypervisor layer runs natively on the physical hardware and enables multiple VMs to run concurrently, sharing the resources of a single physical host. Open source platforms generally support multiple different



hypervisors, popular ones being Citrix XenServer [13] (also used by commercial clouds like Amazon EC2 and Rackspace Cloud) and KVM [14] (part of the Linux kernel). The latter is the default hypervisor in OpenStack and also used throughout this project. A popular interface library used to interact with hypervisors of different kind in a more standardized way is provided by libvirt [15]. Libvirt is used in OpenStack to interface with the KVM hypervisor and as such was used within this work.

The resource pool that makes up the lowest layer consists of all physical machines that are used to host the VM processes that are launched by the upper layers. Even if the individual machines have limited capacities, the upper layers make it look like everything belongs to a one single resource pool.

## 2.2 OpenStack

OpenStack is a popular open source cloud computing platform for building both private and public clouds. It is the platform of choice for companies like Rackspace and HP for their respective public cloud offerings. The software consists of several independently developed software components, organized into a separate subproject for each independent service, providing a wide range of functionality needed to build an IaaS cloud. This most notably include functionality for the provisioning of compute, storage and network resources. The different services interact through well-defined Application-Programming-Interfaces (APIs) exposed as RESTful web services [16]. Under the hood, all OpenStack services are developed exclusively in the python programming language [17]. The project rests on the following principles of openness; open design, open development and open community as stated on the OpenStack website [2]. Any company or individual that wants to join is welcome to do so and after signing a developer agreement, can start contribute code to the project.

### 2.2.1 History

The OpenStack project was initially announced in July 2010 by Rackspace and NASA who also made the initial code contributions. Since then, more than 150 companies and many more individuals have joined the project and take daily part in the ongoing development of this fast moving and increasingly popular cloud platform. The ownership of the project was recently transferred from Rackspace to a newly formed body, the independently operating OpenStack Foundation. The first coordinated software release happened shortly after the projects first public announcement back in 2010 and has since been followed by another six releases; current schedule is one new release every six months.

### 2.2.2 OpenStack Compute

Of the many OpenStack services/projects (the list is growing for every release), only *Compute* is of main interest to this thesis. Compute or “Nova” is the service responsible for providing compute provisioning functionality to a cloud. It can be thought of as a

management layer that runs on top of a free choice of supported hypervisors, exposing a RESTful API for the purpose of provisioning and management. In itself, it consists of a set of service binaries that work together to achieve a common goal. They all interact through messaging and in some cases directly through shared state stored in a central database as shown in Figure 2 below.

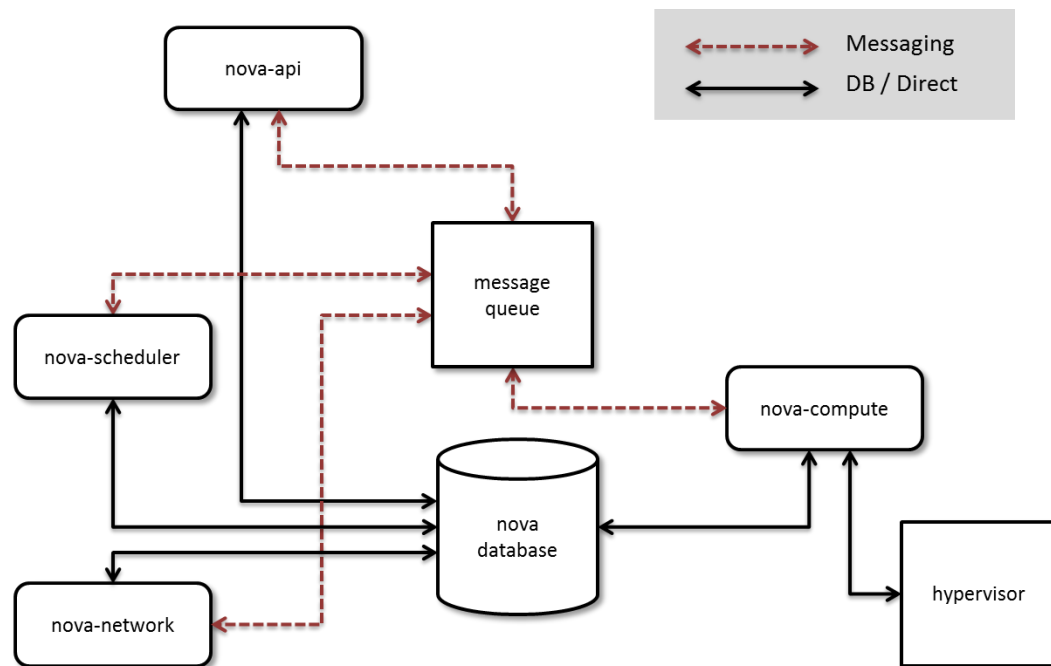


Figure 2: OpenStack Compute architecture overview

To interact with the service, one can target the REST API directly or use the python language bindings provided by the python-novaclient library that also includes a command-line client. Other interfaces like the web based Dashboard, use this and other client libraries to interact with the different OpenStack services. Provisioning requests enters the API and passes initial authorization and verification steps before being sent to the scheduler to decide which one of the available compute nodes should handle the request as explained in section 2.2.3 below.

### 2.2.3 Compute Scheduler

The scheduler's role in Nova is to direct incoming instance requests to suitable compute nodes that in turn will serve the requests handed to them by launching and making available new virtual machines sized according to the specifications of those requests. The scheduler uses a pluggable design, which let it load one of a set of drivers that does placements according to some scheduling policy. The scheduler makes use of host state information, which is sent to it periodically by the compute nodes as the basis of any scheduling decisions it makes.

## 2.3 Cloud Performance Management

The role of performance management of an IaaS cloud can be defined as controlling the resource allocation process so that both customer and provider expectations are met. Customers expect their SLAs to be respected, while the provider, at the same time, must efficiently operate the cloud to fulfill both financial and operational goals.

### 2.3.1 Management Objectives

A service provider defines strategies according to which infrastructure resources of the cloud should be allocated to VM instances of a cloud. Those strategies are expressed as management objectives and are used to instantiate the management controllers that carry out the resource allocation process. A wide variety of management objectives can be defined depending on the type of service being provided, who the customers are, what applications they are going to run, the business model being pursued by the provider, and so on. To fit different use cases, a cloud platform should be flexible enough to support a wide range of such objectives.

### 2.3.2 Constraints

To add more control over what resource allocations are made, different constraints can be introduced that further restrict where a request is placed. Simple constraints like resource constraints are specific to the instance-to-host mapping, while others might involve more than one instance. Constraints belonging to this second category include different kinds of affinity filters, where the placement of existing instances is used to restrict where a new instance is placed, either forcing some instances to be placed the same host or as in the case of anti-affinity, by prohibiting some instances from being placed on the same host.

### 2.3.3 Resource Management

In related work [18], we model a system for cloud resource management as having a set of cooperating controllers. Here, each individual controller has a unique responsibility to fulfill part of a shared global goal, derived from a system-wide management objective used to instantiate all controllers. The different controllers handle:

- Admission control
- Initial placement
- Dynamic placement
- Local scheduling

The work of this thesis is largely what constitutes the initial placement controller of this model, whereas that paper looks at the function of such a system as a whole.

### 3 Related Work

In their paper on Mistral [19], the authors present a system whose goal is to optimize both performance and power efficiency of consolidated VM workloads in a cloud. This is achieved by dynamically adjusting VM placements considering tradeoffs between power, performance and adaptation costs. While the goals of optimized performance and power efficiency are shared with the work presented in this thesis, the means to achieve it are very different and can in fact be used together as complementing techniques. Initial placement on its own can only solve part of the resource management problem and in many cases depend on dynamic placement being part of the full resource management system to add freedom to its placements. Without it, some placements would be considered unsafe, because a change in demand has the risk of overloading a system otherwise.

In [20], the authors present a system that makes use of structural information of an application provided within a request to improve on VM placements, an application being defined as a cluster of related VMs such as a tiered application running on multiple machines and scheduled together as a group. Structural information such as availability and communication requirements of the application complements existing information on resource demands and is used in a hierarchical placement process against a datacenter modeled as a tree structure based on topology and other logical groupings of physical machines. While this approach has the potential of providing very good placements, it depends on information that is generally not available in a public IaaS setting. Here, little to nothing is generally known of the applications that are hosted within the VMs. The work of this thesis, in contrast tries to make the best out of the limited VM information available by instead using extended host information, something over which the provider has full control.

The authors of [21] present a hierarchical resource management system that tries to overcome the scaling issues normally associated with a centralized design while still being able to make fully informed placement decisions based on global state, as opposed to a decentralized system that only act on local knowledge. The problem being researched here is not so much about trying to solve a specific placement problem but rather to make such a system scale to let the algorithms in use work efficiently. While this is not directly related to the problem being looked at in this thesis, scaling issues are certain to exist in the current OpenStack scheduler and to be able to scale well, the ideas presented here could certainly provide value here.

Some existing resource allocation systems that solve problems similar to the ones now faced in cloud computing can be found in the enterprise arena where virtualization have been used extensively to consolidate workloads onto a smaller set of machines. The market leader, VMware with its vSphere product is a prime example, with details of the

products resource management component, known as Distributed Resource Scheduler (DRS), being presented in [22]. While DRS similarly aim at solving the VM placement problem in a virtualized environment, it differs mainly in the system size being targeted. Whereas the problem being researched here is that of cloud scale systems, DRS is currently limited to a setup of at most 32 hosts [23].

Public clouds like Amazon EC2 [1], Rackspace Cloud [11] and HP cloud [12], whether they use proprietary software or open software, for competitive reasons usually don't reveal what VM placement strategies are used in their respective clouds. One can only assume that this is an active area of research since it has a direct influence on the operational costs of their clouds. Compared to their commercial counterparts, public cloud packages usually have rather sparse support for advanced scheduling out of the box, which is something this thesis helps address.

An example of the above is the component in OpenStack responsible for VM placement, called nova-scheduler that comes with implementations for a few algorithms that all provide very basic scheduling capabilities. They are; (1) the chance scheduler that does random placement, (2) the simple scheduler that picks a host with the fewest running instances as its next target and (3) the least cost scheduler that provides a framework for picking a host based on filtering and cost calculations of included hosts. While number of instances running on a host, as done in (2), may be an indication of the actual resource usage of that host, this is hardly the best metric for basing resource allocation on. This thesis aims to improve on that by making scheduling decisions based on actual usage of compute resources (CPU, memory) of the cloud hosts. It will improve on (2) by adding flexibility to what objectives are supported by allowing different objectives to be specified at runtime. It will improve on (3) by extending the existing framework to support additional objectives and will also give the scheduler access to new host state that will help it better understand and be able to predict resource demands as part of the scheduling process.

On the algorithms themselves included in this thesis, an extensive amount of previous work has already been presented on the scheduling problem, and dates back to the late sixties where Graham in [24] first introduced the scheduling problem and among other things suggested greedy algorithms for the online version of the scheduling problem. This work however is not about finding the optimal algorithm to a specific theoretical problem but rather to implement and evaluate a general solution on top of an existing platform, accepting much of the restrictions imposed by it. One of such restrictions is not having extended knowledge about the application of a request, as in IaaS, it is generally the case that very little is known about the application being hosted by the VM and of its future resource demands etc.

## 4 Initial Placement Controller

This chapter presents the architecture and details of a generic initial placement controller for use in an IaaS cloud. Furthermore, it details different management objectives and associated algorithms used to instantiate such controller.

### 4.1 Controller Architecture

Figure 3 shows the overall architecture of the initial placement controller with the different inputs used to decide upon the placement of arriving instance requests. The cloud operator instantiates the controller with the policies used to enforce a specific management objective. When a request arrives, the controller then use information provided within the request together with state information collected from the cloud hosts to decide on a placement that best fulfills the objective. The most common information provided in a request is the amount of compute resources to be allocated for the VM being launched. Other information that might be used to further improve on the placements includes details on the life-span of requests, profiling on the application(s) that is going to run inside a VM and communication patterns between VMs. The information gathered from cloud hosts consists of total resource capacities, the amount of resources currently being allocated to running instances, and actual resource utilizations. The latter could be used for purposes of improving statistical multiplexing to better utilize the resources of the cloud.

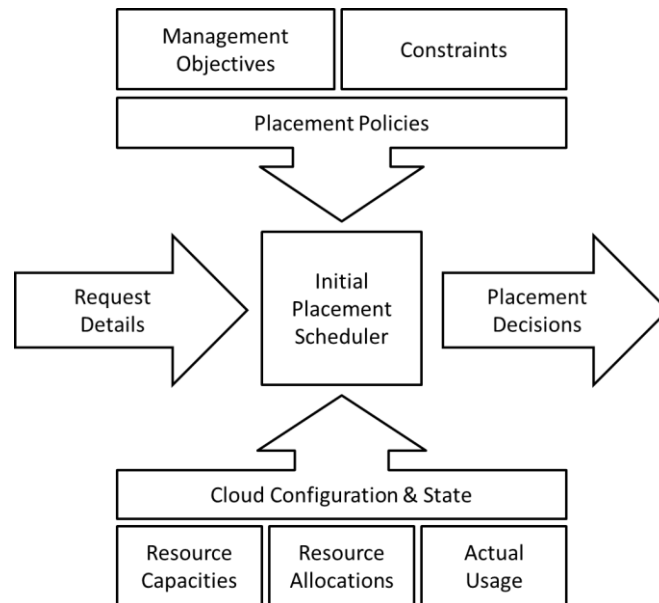


Figure 3: Initial Placement Controller Decision Engine

Each request is served immediately upon its arrival, one-by-one, in a serialized way, without knowledge of future requests. In the simple case, each request is for a single instance, but one could also think of supporting requests for multiple instances that

together constitutes a distributed application of some kind. In this case, information about the communication patterns among VMs could provide valuable hints to the decision engine.

## **4.2 Controller Input**

### **4.2.1 Placement Policies**

Management objectives might change and therefore the initial placement controller is designed to be generic in what objectives to support. Placement policies are the functions that instantiate the controller for a specific management objective. Constraints used to further restrict placements, management objectives and related functions are detailed in section 4.3 below.

### **4.2.2 Request Details**

In the IaaS cloud model, a cloud provider normally does not have control or knowledge of what applications are executed inside a VM. The information provided in the request help the initial placement controller to find the best placement for it. Most commonly, a request includes the amount of resources that should be allocated for the instance (usually what sets the price) and some metadata to describe it. Research shows that by including extended information in a request, such as information on the application being executed [20] or details of the lifetime of a request [25], scheduling can be made with higher precision with respect to the active management objective. As such information is not easily obtainable in OpenStack it is currently not part of the implementation that follows.

Requests can be served in different ways. The easiest and most commonly used is to serve each request individually, but since many applications might be distributed in some way, a better approach would be to treat related requests together as a group. This way, relations between the instances could be used to influence the placements. As an example, two VMs that communicate intensively could be placed on the same physical host to reduce the network traffic being generated in the system. Yet again, this is information on the level of the application that is not available to the scheduler in OpenStack and therefore the simpler model is what is considered for this thesis.

### **4.2.3 Cloud State**

Cloud state is what the instance requests are being matched against to find optimal placements. The resource management system keeps track of the current cloud state and feeds it to the placement controller. Information about capacities of physical machines used to host VM loads and the network that connects them is used together with current VM allocations to find machines with spare capacity where new VMs could run.

By collecting usage statistics for both physical machines and VMs, one could make placement decisions that improve on the way statistical multiplexing is being used to further improve on the utilization of resources of the cloud.

### 4.3 Management Objectives

This section outlines commonly used management objectives, motivations for their use, and what is needed to implement them. For the two objectives considered for implementation in this thesis, specific algorithms to be used are presented in detail. Before going into the actual objectives, a first section touches upon different ways to measure host resource usage, as it is used as input to the algorithms. A concluding section discussed various constraints that can be used in combination with the management objectives to further restrict placements.

#### 4.3.1 Reservation-Based vs. Utilization-Based Scheduling

Resource usage of a host can be measured in different ways. One way would be to look at reservations made for VMs on a host in addition to some resources reserved for the underlying OS/hypervisor itself. Using this scheme, a host would be considered full when resource reservations sum up to the total capacity of that host. The main benefit of this approach is that such reservations don't change over time and that each VM have the full amount of resources allocated to it, available at all times. The downside is that since VMs normally don't demand and use all of their allocated resources at all times, a lot of resources goes unused and therefor wastage is high.

A different approach is to monitor the actual resource demands of running VMs on a host and to make all unused resources available to other VMs through VM multiplexing. This way VMs could be packed tighter and the underlying hardware be better utilized. The major concern here is that this might cause violation of SLAs, since not all VMs running on a host can demand their full resources at the same time. By monitoring usage statistics and having a scheme that includes dynamically moving VMs between hosts, using live migration, to adapt to load changes, this problem can be avoided.

The algorithms presented below are generic and can work with either reservations or actual demand as the measurement of resource usage.

#### 4.3.2 Online Algorithms

The initial placement controller places new VMs on top of the existing placements made. It does not consider moving existing VMs neither does it have knowledge about any future requests. As such it tries to find the best placement for each individual request one-by-one as they arrive. The class of algorithms this usually refers to are called online algorithms and their properties have been studied in detail in [26], [27], and [28]. The focus here is not so much to find the optimal algorithm for doing initial



placement, but rather to study how the controller can make use of such algorithms to improve the overall performance according to some management objective.

### 4.3.3 Balanced Load Objective

A very common objective that is supported in some form by most IaaS software platforms is balanced load. The objective is to place VMs in such a way that the load is balanced across all physical hosts.

Different strategies can be used when placing VMs to achieve this objective. One strategy would be to place VMs so that the load variance across servers is minimized. Another strategy would be to place VMs so that the maximum load of servers is minimized. The above strategies make use of the current state of servers to decide on a placement. Other means of achieving balanced load include simpler approaches like random and round robin schemes that make no use of state information for placing VMs.

The algorithm presented below uses the second approach and tries to minimize the maximum load of the servers in the cloud. It does so while considering two kinds of resources; CPU and memory. While it would be a much simpler problem to balance load using just one kind of resource, this now also introduces the problem of effectively utilizing the combined resources of individual servers, so that resource waste is minimized.

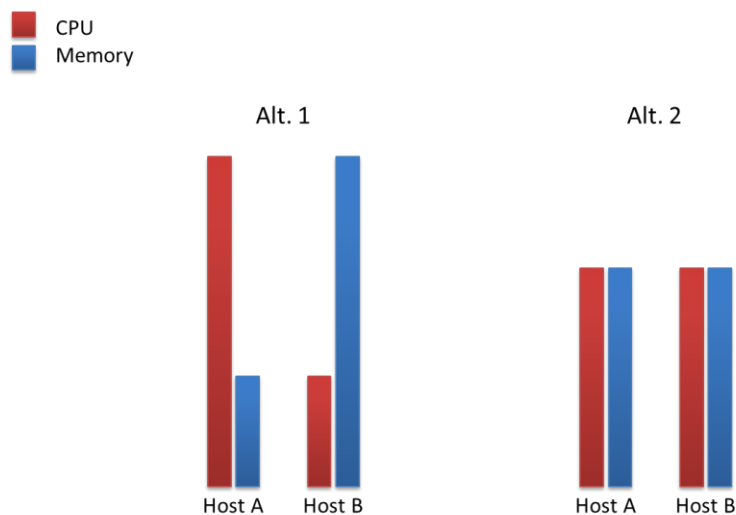


Figure 4: Ways to distribute the same load across two servers

Figure 4 shows two ways to distribute the same load on two different hosts. Both ways can in some sense be considered to result in a balanced load since the total resource consumption in both cases is equally balanced across the two servers. Now, alt. 2 also balances the individual resources, meaning that there is minimal waste (not hitting the max capacity of one resource while the other being mostly unused). Of course it is not

always the case that the resource usages of both resources are globally the same like in the example. In the case when one resource is globally more constrained than the other, it still makes sense to minimize the waste to the degree possible. To achieve the above, the algorithm used to calculate the cost of placing VMs is defined as follows.

Assume a VM  $m$  is to be scheduled. Let  $U_{cpu(n,m)}$  and  $U_{mem(n,m)}$  be the (normalized) CPU and memory utilizations of host  $n$  when  $m$  is placed on it. Then the cost  $C_{(m,n)}$  to place VM  $m$  on host  $n$  is calculated as the maximum of the two.

$$C_{(m,n)} = \max(U_{cpu(n,m)}, U_{mem(n,m)})$$

#### 4.3.4 Energy Efficiency Objective

Datacenters are big consumers of energy. As energy costs are rising and quickly becoming a major part of the total datacenter budgets, there is a growing interest in reducing the energy draw in the datacenters. This can be achieved through server consolidation and the objective here is then to place VMs on as few hosts as possible while satisfying their resource demands, thereby allowing unused hosts to be put into standby or other low power modes to save energy. To realize the actual power savings, this scheme must be combined with power management software that controls the power states of machines, otherwise the objective would be resource maximization.

Other means to reduce total power consumption of a datacenter include efforts to reduce power draw of individual servers by locally scheduling tasks to make efficient use of processor support for distributed voltage scaling (DVS) [29].

The problem of server consolidation is identical to the bin packing problem where items of different sizes are to be packed in as few bins as possible [30]. For the online version of the problem, several different strategies exist. Simple ones include next fit, first fit, and best fit. Next fit considers one bin at a time and fills it as much as it can before moving to the next bin, never going back to a previously filled bin. First fit always starts at the first bin and places the next item in the first bin that has enough room for the item. Best fit always considers all bins and places the next item in the bin that has the smallest room left after placing it there.

The solution presented below uses best fit. Like previously, with balanced load, it considers two kinds of resources; CPU and memory. The cost of placing VMs is defined as follows:

Assume a VM  $m$  is to be scheduled. Let  $U_{cpu(n,m)}$  and  $U_{mem(n,m)}$  be the (normalized) CPU and memory utilizations of host  $n$  when  $m$  is placed on it. Then the cost  $C_{(m,n)}$  to place VM  $m$  on host  $n$  is calculated as the maximum remaining capacity of the two.

$$C_{(m,n)} = 1 - \min(U_{cpu(n,m)}, U_{mem(n,m)})$$

#### 4.3.5 Minimize Network Bandwidth Objective

For many different reasons, applications can be distributed over several machines, like a three-tiered web application having a database layer, an application layer, and a presentation layer. By utilizing this knowledge, virtual machines that need to communicate extensively can be placed near each other, possibly even on the same physical machine, to minimize the bandwidth needs of a shared network infrastructure.

Another similar but slightly different objective, is to speed up communication between different parts of an application by reducing network latency. This is achieved by placing the VMs of an application so that the number of network hops between them is minimized.

To support such objectives, the scheduler would need to have detailed knowledge of application structure and expected bandwidth requirements between different parts of it. Furthermore, the scheduler would need to know the topology of the physical network and in addition, total capacities and current usage levels of individual network links to match this against. Finding an applications optimal placement thus becomes a graph matching problem.

Detailed application knowledge could easily enough be added to requests, but for scheduling to be effective, would likely require multiple VM instances to be requested together as a group.

Network topology and link properties would need to be collected and stored so that scheduling could take advantage of them. Currently, scheduling is limited to state reported from compute hosts and some effort would be needed to let the scheduler access data fed from an external source. Additionally, as networking in OpenStack has been split out of Nova and now lives in a separate project, this work might need to span several OpenStack projects.

As this thesis only considers computational resources, and also because the necessary application knowledge is not easily obtainable in the general IaaS model used by OpenStack, this objective has not been considered for implementation.

#### 4.3.6 Fair Allocation Objective

The resources of a cloud are commonly used by more than one tenant and since virtualization is being used, VM workloads from different tenants are possibly multiplexed on the same physical hardware. In the case where a cloud provider allows for oversubscription of the cloud resources, there is always a risk that performance of VMs belonging to one tenant is negatively impacted by resource hungry VMs belonging to other tenants. One possible solution would be to reserve individual hardware for use by a single tenant, but this would come at the cost of missing out on flexibility of using a shared pool of resources. Another approach would be to make sure allocations are fairly

distributed among tenants. While the first approach can be implemented using VM placements alone, the second one is more about controlling local scheduling of resources at the hypervisor layer of individual hosts. This is typically the domain of either the OS or hypervisor scheduler, and as such, this objective has not been considered for implementation in this thesis.

To support such an objective, methods for controlling local scheduling in either the OS or hypervisor would need to be exposed by compute drivers in OpenStack so that the scheduler could make use of them. To control for example CPU entitlement, one way would be to use CPU affinity as a tool to let tenant VMs be separated by physical CPUs. Another way, which could be used to control other resources and I/O as well, would be to use the Linux kernel feature known as cgroups.

#### **4.3.7 Service Differentiation Objective**

For various reasons, IaaS providers may want to offer a differentiated service to their customers. Some batch type workloads do not require the same guarantees as an interactive web application and can therefore be offered at a lower price tag. For the provider, this has the added benefit of allowing high guarantee VMs to be multiplexed with lower guarantee ones such that when resources are constrained, some VMs can be slowed down or even completely paused to make room for higher priority ones until more resource becomes available.

While part of this problem involves placing VMs optimally, there is also a certain amount of local scheduling involved, similar to the fair allocation objective described above. Due to the same reasons, such objective has not been considered for this thesis, and to implement it, the same control mechanisms for controlling local scheduling would need to be made available for this as well. Additionally, since this would be a feature exposed to the cloud consumer, an API extension would need to be added to open up the ability to set the service class of an instance.

#### **4.3.8 Constraints**

To restrict VM placements beyond what is defined by a management objective, various constraints can be enforced during scheduling. Constraints can broadly be categorized as either (1) instance-to-host or (2) instance-to-instance constraints. For the first category of constraints, the term host can mostly be replaced with any other physical or logical entity like network, subnet or cloud and so on. Different resource constraints fall into this category; for instance, a hypervisor used to host VM processes is limited by the amount of memory available on the physical machine it runs on. Other constraints that fall into this category are licensing constraints, where in some cases only a certain number of operating system or application licenses are allowed to run within a domain.

Constraints belonging to the second category are the class of affinity constraints that restrict what instances must or must not be hosted within the same specified boundary,

whether that is a physical host, a group of hosts, a subnet or a network. Many cloud systems allow for hosts to be grouped by region and/or availability zone. The former representing geographic locality, while the latter represent some shared infrastructure like network connectivity or power distribution, where a group of hosts share a single failure domain of some kind. Other constraints that belong here are the different types of network constraints that are used to provide bandwidth guarantees or that limits allowed latency between instances.

## 5 Implementation in OpenStack

Like many components in OpenStack, the scheduler component features a pluggable architecture that allows for easy replacement of the scheduling mechanism as well as development of new schedulers to complement the ones already there. In choosing between writing a new scheduler driver from the grounds up and extending an existing one, this thesis chooses the second approach as explained in detail in section 5.1.

To achieve the goals of the proposed design, changes to not just the scheduler driver, but also parts of the Nova codebase responsible for feeding host state to the scheduler are needed as follows. The nova-compute service and the virtualization driver it uses, and likewise the database, need modifications to handle extended state needed by the scheduler as explained in section 5.2.

To measure and evaluate the performance of the controller, a separate monitoring system that allows for both real-time and historical analysis of key performance metrics are implemented as well. Details of it are found in section 5.3.

In summary, the implementation has the following three deliverables:

- A scheduler driver that builds on top of and extends the functionality of the least-cost scheduler in OpenStack.
- An extension to the current host state reporting mechanism to continuously feed the scheduler with statistics on host resource utilizations.
- A performance monitoring system used to collect and visualize host performance data for evaluation and demonstration of the scheduler.

### 5.1 OpenStack Compute Scheduler

Implementation was carried out using the latest available stable release version of OpenStack that at the time the thesis implementation began was the “Diablo” version. This version introduced some new scheduler techniques, explained in detail below that was reused in the new scheduler being built.

#### 5.1.1 Brief History of the Compute Scheduler

The compute scheduler in OpenStack, like most parts of the code, has evolved for every new release shipped. Releases prior to Diablo only shipped with very basic scheduling capabilities, limited to one of two scheduling drivers; (1) the chance scheduler that does random selection and (2) the simple scheduler that does load balancing in the cloud by placing an equal number of instances on each host.

The Diablo version introduced the first try at a modular scheduler driver, called the least-cost scheduler that has a framework for plugging different scheduling policies into it to change its behavior. This scheduler also turns scheduling into a two-step process,

known as filtering and weighting. Here, the list of candidate hosts for a request is first passed through a configurable filter to single out only those hosts that pass filter checks. Filters are used to eliminate hosts not able to serve a request due to resource constraints or some other capabilities not being met. Following this, the remaining list of hosts is passed on to weighting, where costs are calculated for hosts that remain. The costs, calculated as the sum of a configured set of cost functions for each host, are then used to rank hosts according to their suitability of handling a request. Finally, the request is forwarded to the highest ranked host, i.e. the one with the least cost, hence its name. The OpenStack Team at IBM has written an excellent article that explains the details of how the least-cost scheduler works [31].

At the time scheduler implementation of this thesis begun, the upcoming “Essex” version was still in its early stages of being developed. Still, some improvements that had been incorporated in the least-cost scheduler made this version of the driver a favorable target for extending upon. As such, this version was chosen as a starting point for a new scheduler and was back-ported to work in the Diablo version of OpenStack that was what the testbed otherwise used.

### **5.1.2 Overview of the nova-scheduler service**

The OpenStack Compute scheduler, known as the nova-scheduler service, is responsible for mapping instance requests onto physical hosts called compute nodes. Compute nodes in turn run the nova-compute service on top of a supported hypervisor. When the scheduler service launches, it loads a scheduler driver that contains the actual scheduling logic. A scheduler driver is any class that derives from a base driver class and implements its interface.

A number of simple scheduler drivers are included in the package. More advanced filters can be written as long as they implement the required interface of the base driver class. The base driver, in addition to defining the interface requirements, also holds some of the basic necessities needed by every scheduler. This includes easy access to the global system state and some utility methods used by most schedulers. The class diagram in Figure 5 details the relation between the scheduler driver classes in OpenStack Diablo.

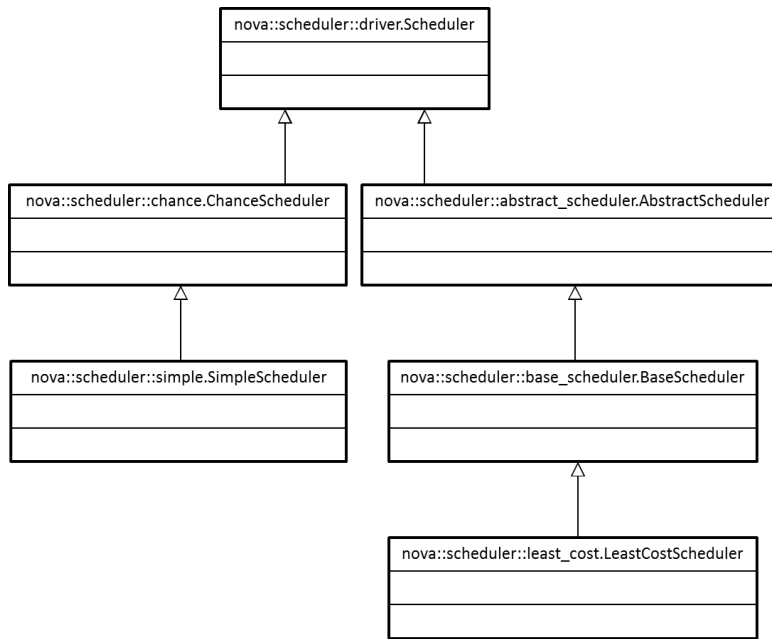


Figure 5: Diablo scheduler driver classes

On top is the base driver that every other scheduler must inherit from. The schedulers to the left are the most basic ones that do random placement and instance counting. The ones to the right together add up functionality in the least-cost scheduler at the very bottom. The abstract classes in the middle were merged in Essex to become a single class. The simpler code that this resulted in was one of the reasons the Essex development version of this driver was chosen to extend upon.

### 5.1.3 Extended Version of Least-Cost Scheduler

The algorithms proposed in chapter 4 are designed to find placements that minimize the cost in respect to a specific management objective. This same approach is what the least-cost scheduler use, which made it a good starting point for building a new scheduler. A view of how the request flow in the least-cost driver looks like is shown in Figure 6 (blue color indicates parts of the flow that were modified).

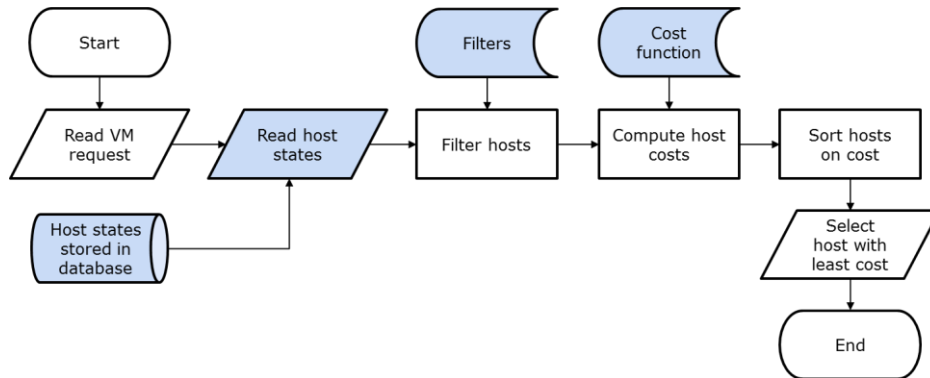


Figure 6: Request flow of the least-cost scheduler



To build scheduling logic that uses state information (host capabilities, host resources allocated etc.) to decide where to place new instances, most of what is required is to build the filters and cost functions that implement the scheduling policy that one wants.

For this implementation, besides the porting that was necessary to have the Essex based driver run in a Diablo setup, new filters and cost functions are mainly what were added. In addition, only some minor changes to the code that read host states from the database was needed to support getting new state information stored in the database.

To support both balanced load and energy efficiency objectives, two new resource filters was added. These filters would look at the actual resource usage of either CPU or Memory as reported by the host to decide whether a request would fit within some configured maximum limit. For the balanced load objective, such filters are there to keep the host from becoming overloaded. As balanced load tries to distribute the load evenly among hosts, this will mostly be an issue when resource usage of the entire cloud nears its limits. However, for energy efficiency objective that tries to consolidate load, these filters are what decides when a host is fully packed and when a new host should be targeted.

The objectives themselves and their algorithms are otherwise implemented as cost functions, to be used by the least-cost scheduler. Here one cost function for each objective implement the algorithm associated with it.

## 5.2 Utilization Reporting

As previously mentioned, the scheduler implementation in OpenStack Diablo has limited access to host state to use in the scheduling process. Also, the drivers used by the compute service to connect to different hypervisors, at the time had varying support for collecting such state and send it back to the scheduler, meaning that some schedulers, specifically the least-cost scheduler, were in fact only supported if using a specific hypervisor (XenServer).

To support scheduling based on actual resource usage, state reporting capabilities in the compute service would need to be extended to periodically report and store information on the actual resource usage of hosts. The obvious place to store such extended information is the existing location in the database where existing state, such as total resource capacities of hosts, are already kept.

The implementation as such consists of two parts; (1) extensions to the `compute_nodes` table to support storing of the additional state and (2) extensions to the `periodic_tasks` method of the compute manager class plus within the libvirt driver used to interface with the KVM hypervisor, to read and populate the added database fields at a regular interval.

## 5.2.1 Database Extensions

OpenStack Compute uses a common database to store state for all its services. A dedicated table named `compute_nodes` is used to store host state sent from all compute nodes at a periodic interval. The state is then used by the scheduler as input for the placement decisions it makes. Figure 7 shows the extended schema for the `compute_nodes` table that was used to store the additional information related to actual utilization of host resources.

| Field                           | Type                    | Null | Key | Default | Extra          |
|---------------------------------|-------------------------|------|-----|---------|----------------|
| <code>created_at</code>         | <code>datetime</code>   | YES  |     | NULL    |                |
| <code>updated_at</code>         | <code>datetime</code>   | YES  |     | NULL    |                |
| <code>deleted_at</code>         | <code>datetime</code>   | YES  |     | NULL    |                |
| <code>deleted</code>            | <code>tinyint(1)</code> | YES  |     | NULL    |                |
| <code>id</code>                 | <code>int(11)</code>    | NO   | PRI | NULL    | auto_increment |
| <code>service_id</code>         | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>vcpus</code>              | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>memory_mb</code>          | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>local_gb</code>           | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>vcpus_used</code>         | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>memory_mb_used</code>     | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>local_gb_used</code>      | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>hypervisor_type</code>    | <code>text</code>       | NO   |     | NULL    |                |
| <code>hypervisor_version</code> | <code>int(11)</code>    | NO   |     | NULL    |                |
| <code>cpu_info</code>           | <code>text</code>       | NO   |     | NULL    |                |
| <code>cpu_utilization</code>    | <code>int(11)</code>    | NO   |     | NULL    |                |

Figure 7: Modified database schema of the `compute_nodes` table

The rows marked in yellow are the ones used to store extra state used by the extended scheduler. Of the two, `memory_mb_used` existed previously, but was not previously used due to lack of support within the libvirt driver. In fact, none of the values in this table were updated periodically, if used with the libvirt driver. Using libvirt, the table was populated once at service startup and when manually invoking commands in preparation for doing a VM live-migration.

The `cpu_utilization` field was added to track actual host CPU utilization in addition to the existing information on the number of vCPUs allocated to VMs on the host.

## 5.2.2 Driver Reporting

The libvirt driver implementation was extended with two new methods to collect host CPU utilization. The first method sampled the CPU utilization and calculated a moving average over samples from the last minute (configurable). It was called periodically and used the OS /proc interface to collect CPU utilization values from the host. The second method was added to solve a problem that arose when scheduling new VMs based on actual usage. After a VM has been scheduled to start on a host, it will take some additional time to build its environment, before it can be powered on. This introduces a delay in the feedback loop back to the scheduler, since the new VM will not add to the utilization of resources until it is actually started. If additional VMs are to be scheduled within this time period, they will all be scheduled on the same host because the

calculated host cost would not have changed between requests. To solve this, the added method temporary reserve resources for new VMs being built, until the time they are started and actually use resources.

### 5.3 Performance Monitoring

OpenStack like most other cloud platforms exposes its core management APIs as a set of RESTful web services. In addition, command line tools and programming libraries that use these APIs are the ones being used for provisioning VMs and other management tasks. Furthermore, a web based interface called the Dashboard that also builds on top of these APIs provides for a more user-friendly management experience in dealing with the cloud. The exposed APIs are primarily aimed at providing consumer functionality like VM provisioning, user management etc. Administrative control and monitoring is currently not available, nor is it provided in the Dashboard.

#### 5.3.1 Zabbix Monitoring Package

To provide monitoring capabilities in OpenStack, an open-source monitoring package called Zabbix was used. A Zabbix installation consists of a monitor agent installed on each system being monitored and a server component that collects and stores data for monitored items in a central database. In addition, an included web application can be used to administer the system and to view data for items being monitored. Data can be viewed using periodically updated time-series graphs that are generated as images of fixed layout.

#### 5.3.2 Custom-Built User Interface

The fixed layout of the graphs produced by Zabbix didn't provide the necessary flexibility to be considered useful, and therefore was not used. Instead a custom PHP [32] web application was developed that would read data directly from the Zabbix database and present it in a more suitable way. To generate time-series graphs out of Zabbix data, flot [33] was used by the developed web application.

The monitor interface of the developed web application, of which a screenshot is shown in Figure 8, used the same basic layout as the interface provided in Zabbix. Like the Zabbix interface it used the concept of screens to group multiple graphs and present them side-by-side, a screen being nothing more than a landing page that would load additional HTML content in sub-frames. This way, graphs would have the ability to be shown together as part of the screen or individually since each graph would in itself be a HTML page. A graph would consist of two individual PHP scripts; one script that would load and display in the browser and one that would serve the actual data from Zabbix in response to repeated AJAX [34] requests initiated by the first script. This would provide for a smooth user experience without complete page reloads to fetch new data. What data and graphs the page would show, like items in a graph, coloring of graphs, stacking

etc. was configured separately in a configuration file read by the page in response to the id specified in the page request.

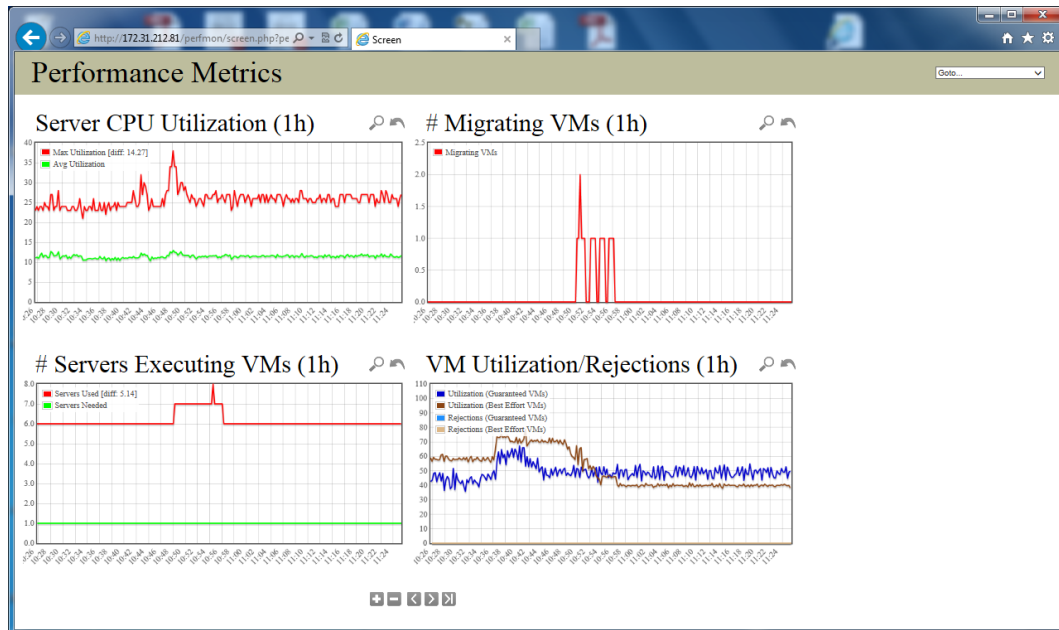


Figure 8: Screenshot of the custom-built monitoring interface

### 5.3.3 Zabbix User-Defined Monitoring Items

Out of the box, Zabbix includes predefined monitor items for a wide range of systems and metrics. In addition user-defined items can easily be defined and stored in an agent configuration file. A user-defined item is simply a script or command that the Zabbix agent would execute to return a single value that then would be sent to the Zabbix server.

User-defined items were mostly used to collect the metrics from compute nodes in this setup. Host statistics like CPU and memory usage were collected directly from the OS (/proc). VM statistics were collected using either libvirt or by monitoring the KVM processes directly with the built in ps command. Aggregates for values collected across all compute nodes were defined to produce sums, averages and maximum/minimum values as needed. Zabbix external checks, a way to collect items directly at the Zabbix server, were used to execute scripts for more advanced calculations not supported within Zabbix and for queries targeted directly against the Nova database in OpenStack.

## 6 Scheduler Evaluation

Part of the thesis is to evaluate the performance of the implemented scheduler. In doing so, a functional cloud setup was used to perform experimental evaluation of the scheduler under various conditions. This chapter describes the testbed configuration and further details the metrics that was collected and the experimental settings used for individual tests.

### 6.1 Testbed Details

At the heart of the cloud testbed, was the OpenStack based cloud setup that was installed on a group of powerful servers in a controlled laboratory environment. In addition to this, the testbed also included means to generate load in the system and monitoring software used to collect performance data out of the system during tests.

#### 6.1.1 Cloud Servers

The hardware used for the testbed setup consisted of nine identical high performance Dell PowerEdge servers, each equipped with dual 12-core AMD Opteron processors running at 2.1 GHz speed and 64 GB of RAM. The servers were connected using one Gigabit Ethernet switch for VM traffic and one 100 Mbit Fast Ethernet switch for cloud management and monitoring traffic. Each server used Ubuntu Server 10.04 LTS as the operating system with version 3.0 of the Linux kernel installed on a single 500 GB SATA disk.

#### 6.1.2 OpenStack Cloud Setup

The alternatives for setting up an OpenStack cloud range from installing a development environment using the latest source version directly from the OpenStack repositories to installing a production environment using stable release versions, usually from packaged versions of the components like the ones provided by the Ubuntu package repositories. One must also decide on what components to install depending on the required functionality of the cloud being built.

For this cloud, to serve multiple purposes as both a development/test environment and an evaluation/demo platform, the choice was ultimately made to install from packages. The latest available version at the time was the “Diablo” version (2011.3). The OpenStack software packages included in this setup was:

- Compute (Nova)  
*Compute sub-services: nova-api, nova-network, nova-scheduler, nova-compute*
- Image Service (Glance)
- Dashboard (Horizon)
- Identity Service (Keystone)

The setup had one machine acting as the cloud controller, running all sub-services except nova-compute, which was running on the remaining eight machines that acted as compute nodes. The cloud controller in addition to OpenStack components was also running additional required software, including a MySQL database server, a RabbitMQ message queuing server and an Apache web server. On the compute nodes, KVM and libvirt were used as the hypervisor respective hypervisor management interface through which nova-compute would issue its VM commands.

### 6.1.3 VM Image Configurations

During evaluation, two base images were stored in Glance and used to launch VMs based off, to generate actual load in the system. The images contained preconfigured operating systems and applications for two different applications. One image contained an interactive web site, based on Joomla [35]. This image included an HTTP client, used as request generator and configured in such a way that the VM on average would have a resulting CPU utilization of around 70%. The other image was configured to launch a compute intensive application, which would spawn worker processes equal to the number of virtual CPUs configured for the VM to maximize its load. The task of the application was to calculate the decimals of  $\pi$  as the duration of its lifetime. The memory footprint of the two would differ in that the former would make the VM use somewhat below 2 GB of RAM, while the latter would have a really small memory footprint, making the VM use less than 512 MB of RAM.

### 6.1.4 Instance Request Generation

Due to lack of publically available traces showing real instance request traffic to a production cloud, the evaluation was instead based entirely on requests being synthetically generated. For this, a simple tool was written in Python using the client bindings provided as part of OpenStack (python-novaclient) to interact with the cloud testbed, issuing commands like launching and terminating instances. The tool used a simple command-line interface for specifying parameters to adjust the way instance requests were generated.

The tool was written with the ability to both re-play existing traces of events, and to generate random events using exponential distributions with set inter-arrival times of instance requests and life-times of created instances. Of the two methods, only the random method was used however. For some tests that used very long inter-arrival times, the warm-up period necessary to fill up an initially empty cloud with running instances to reach a steady state, would take very long time (days!). Therefore, the tool was equipped with the option to speed-up time between arrivals during warm-up, allowing it to reach steady state in minutes, independent of inter-arrival times used for the test.

### 6.1.5 Zabbix Monitoring Setup

Zabbix server software and its accompanying database used to store collected data were installed on the cloud controller while Zabbix agents were installed on all compute nodes. Agents were configured to use user-defined monitor commands for collecting metrics as defined in section 6.2.1.

## 6.2 Cloud Performance Metrics

Cloud performance is defined here as fulfillment of the currently active management objective and as such, it is measured differently for each objective. Since we are measuring the performance of the cloud as a whole, we make use of aggregates of individually collected host metrics. Achieved results are then compared with the optimal case, or rather an approximation of the optimal case as explained in 6.2.2 and 6.2.3.

### 6.2.1 Individually Collected Host Metrics

The following host metrics were collected at regular intervals on each host:

- *Host CPU Utilization*  
Physical CPU utilization of host as a real number between 0 and 1.
- *Host Memory Utilization*  
Physical memory utilization of host as a real number between 0 and 1.
- *Host Max Utilization*  
The maximum utilization of *Host CPU Utilization* and *Host Memory Utilization* above.
- *Host Active*  
A discrete value to indicate VMs on a host. 0 if none active, 1 otherwise.

### 6.2.2 Aggregated Metrics for Balanced Load Objective

To measure cloud performance under the balanced load objective, the following aggregates were calculated from the individual host metrics defined in 6.2.1:

- *Maximum Utilization*  
Aggregate maximum of host metric *Host Max Utilization* over all hosts
- *Average Utilization*  
Aggregate average of host metric *Host Max Utilization* over all hosts

Of the two, *Maximum Utilization* indicates the actual system performance, while *Average Utilization* indicates the target or optimal performance. The difference between the two is then considered to be the scheduler overhead. It is worth to note that *Maximum Utilization* over all hosts never can go below *Average Utilization*, but that in a perfectly balanced system where all hosts are equally utilized, the values of both would become the same.

The *Average Utilization* used to indicate the optimal performance is really an approximation, since it is not guaranteed that for all VM loads, utilizations can be spread equally across hosts. Additionally, this value will in fact under nearly all circumstances be a bit below the actual optimal, meaning that the actual overhead, calculated as the distance between the two measurements, is nearly always slightly less than what the measurements will indicate.

### 6.2.3 Aggregated Metrics for Energy Efficiency Objective

To measure cloud performance under the energy efficiency objective, the below aggregates were calculated from the individual host metrics defined in 6.2.1. To complicate things a bit further, the developed scheduler has cloud configuration parameters; *CPU Usage Ratio* and *Memory Usage Ratio* that controls the limits on how much resources are allowed to be used on hosts. For one of the defined metrics, the configured values must be taken into account to yield correct values.

- *Number of Hosts Used*  
Aggregate sum of host metric *Host Active* over all hosts.
- *Number of Hosts Needed*  
Maximum of the following two aggregates, rounded up to nearest whole number:
  - Aggregate sum of host metric *Host CPU Utilization* over all hosts divided by cloud configuration parameter *CPU Usage Ratio*.
  - Aggregate sum of host metric *Host Memory Utilization* over all hosts divided by cloud configuration parameter *Memory Usage Ratio*.

Like previously in 6.2.2, the first metric is used to indicate actual system performance while the second is used to indicate optimal performance. Likewise, the second is really an approximation that in this case sometimes has the opposite effect in that it yields values above the real optimal value, meaning that actual scheduler overhead sometimes is larger than indicated by the measurements. The reason for this can be explained using the following simple example:

*Of a total of eight running compute nodes, nodes one to seven are currently hosting VMs. The total VM load is such that it could be further consolidated to reduce that to just six hosts. In this case, host number eight and yet another host (undecided which) are unneeded and can be put to standby. Their base utilizations (resources needed by the OS itself etc.) are not needed and should therefore not be included the calculations of the number of hosts needed.*

Considering that the total resources of the hosts are very large compared to their base utilizations, this mostly has no effect simply because the aggregate sum being calculated is anyhow rounded up, i.e. if calculations show 5.75 hosts are needed, the real number



might be a bit less, say 5.65. In both cases though, rounding up to the nearest full number results in the same number of hosts being needed.

### 6.3 Experiments

The instances used for all experiments were generated equally based off the VM images described in section 6.1.3. Instances were created with capacities according to the VM flavor configurations shown in Table 1. Configurations were selected randomly by the generator script with probabilities inversely proportional to the number of cores.

Table 1: Images and VM configurations used

| Image Type                              | VM Flavor Configurations |
|-----------------------------------------|--------------------------|
| Joomla<br>(Interactive web application) | 1 core, 1 GB RAM         |
|                                         | 2 cores, 2 GB RAM        |
|                                         | 4 cores, 4 GB RAM        |
| Pi<br>(CPU-intensive batch job)         | 1 core, 256 MB RAM       |
|                                         | 2 core, 256 MB RAM       |
|                                         | 4 core, 256 MB RAM       |

Each experiment used the same synthetic model to generate the cloud loads for its tests. The model produced VM request arrivals at a constant average rate for the duration of a whole test using a Poisson process. VM arrival rates were matched with VM lifetimes so that the cloud after initial warm-up, on average would have some 80 running VMs at all times. With the VM images used and the load that they generated, the resulting total resource utilization for the entire cloud averaged around 50 % for all tests, though with big variations over time.

Experiments were conducted as executions of three consecutive test runs, averaging the results. Starting from an empty cloud with no running instances, the instance generator script was started with appropriate settings for the experiment. Prior to any measurements, a warm-up period was allowed to pass, having the cloud reach a steady state after which the number of running instances would stay approximately constant over time for the remainder of the test run. The average number of running instances after warm-up, of which is dictated by the rate and lifetimes of instances being created. Each test was run for 4 hours with measurements taken every 30 seconds and test results being calculated as averages over all measurements taken for the full test run.

Each experiment was carried out according to the same process with different VM arrival rates and VM lifetimes according to Table 2 below.

Table 2: VM arrival rates and lifetimes used

| Average VM Arrivals | Average VM Lifetimes |
|---------------------|----------------------|
| 30 s                | 2.400 s              |
| 60 s                | 4.800 s              |
| 600 s               | 48.000 s             |

The obtained numbers indicate similar results for all tested arrival rates. Because of this, only results for a single rate per experiment will be presented below. When deciding on the different arrival rates to use for the experiments, some preliminary test runs indicated that there was a lower boundary for which the scheduler would work. Arrival rate settings below 10-15 seconds showed a rather high failure ratio for VMs being launched which rendered results useless. Limitations of the current scheduler design are further discussed in section 7.1.

### 6.3.1 Experiment 1: Balanced Load Objective

This experiment evaluates the performance of the cloud when targeted with the balanced load objective. The graph shown in Figure 9 below is a screen capture from the demonstrator during one of the test runs. It shows how the measured host utilizations as defined in 6.2.2 changes over time for a total window of 4 hours. The x-axis shows time and the y-axis shows host utilization. The red line shows achieved performance and the green line shows targeted performance. The distance between the two is the performance gap compared to the optimal. For the entire duration shown, the gap remains roughly the same, indicating constant performance throughout the test.

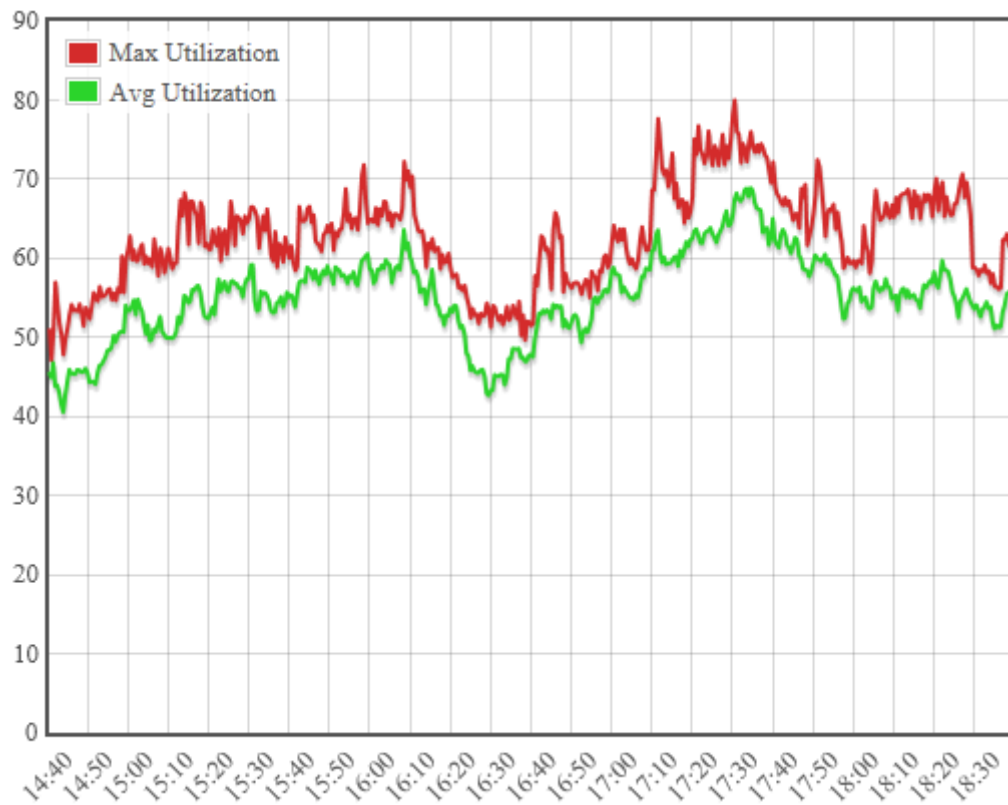


Figure 9: Balanced Load performance for the duration of a test run

Figure 10 shows the summarized results of the entire experiment. On average, the achieved performance is within 10 % of its optimal which is well within the expected outcome.

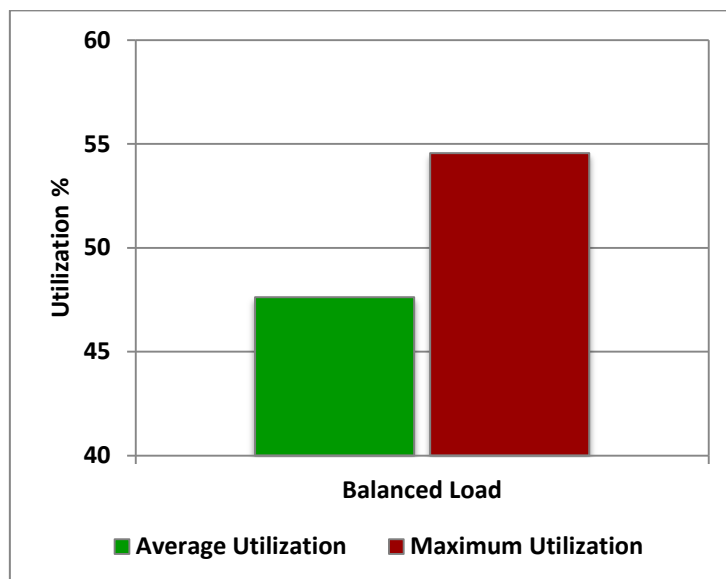


Figure 10: Summarized results for Balanced Load objective

### 6.3.2 Experiment 2: Energy Efficiency Objective

This experiment evaluates the performance of the cloud when targeted with the energy efficiency objective. The graph shown in Figure 11 is a screen capture from the demonstrator during one of the test runs. It shows how the measured number of active/used hosts as defined in 6.2.3 changes over time for a total window of 4 hours. The x-axis shows time and the y-axis shows number of host used/needed. The red line shows achieved performance and the green line shows targeted performance. The distance between the two is the performance gap compared to the optimal. For the entire duration shown, the gap remains roughly the same, indicating constant performance throughout the test.

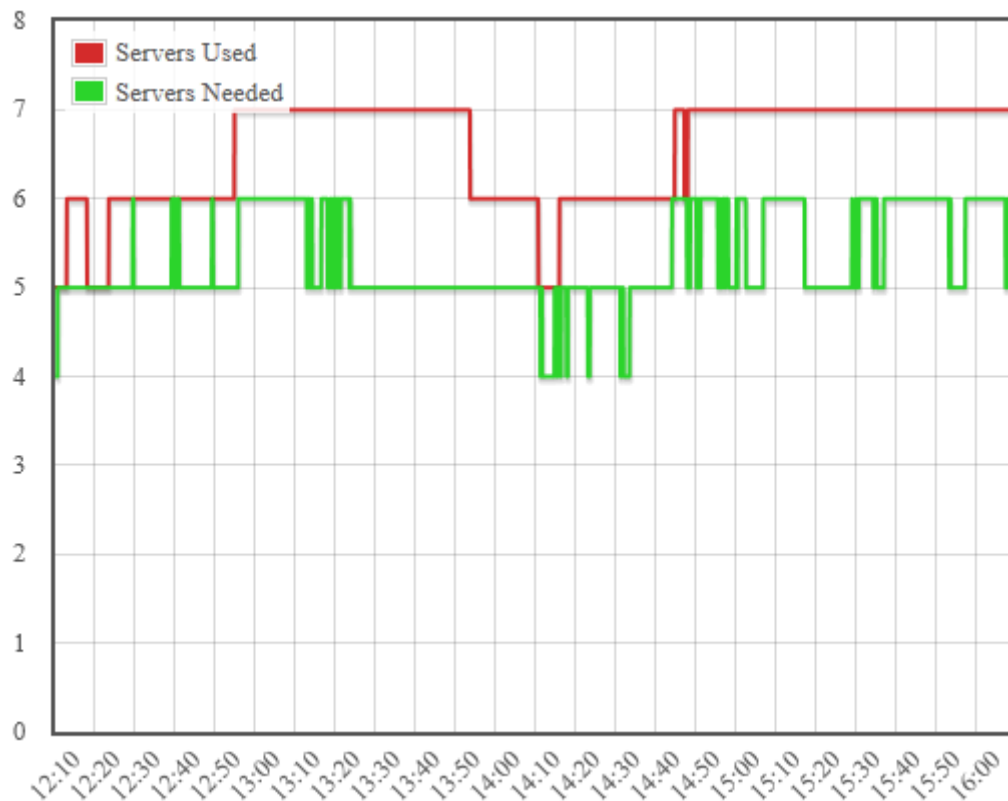


Figure 11: Energy Efficiency performance for the duration of a test run

Figure 12 below shows the summarized results of the entire experiment. On average, the achieved performance is only one host above the optimal which is well within the expected outcome.

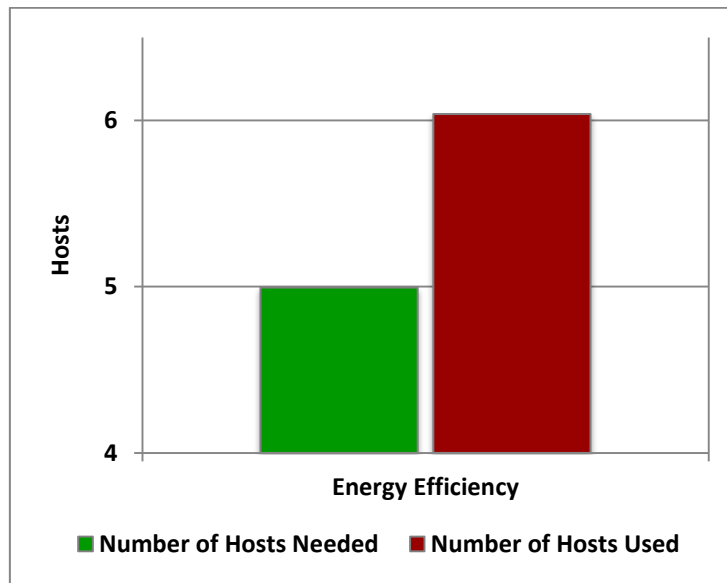


Figure 12: Summarized results for Energy Efficiency objective

## 7 Results Discussion and Future Work

This chapter summarizes the thesis by comparing the thesis objectives to what was actually achieved. Results obtained from performance evaluation are analyzed and conclusions drawn. Lastly two sections are devoted to project experiences and areas that could be considered for future work.

### 7.1 Conclusions

In this thesis, the objective was to implement, evaluate and demonstrate schedulers for OpenStack. This was done by first conducting research on different scheduling problems and strategies related to cloud computing and furthermore by gaining an understanding of the OpenStack Compute architecture and its scheduler component. After this, a working testbed built using OpenStack was set up and used to implement a scheduler for initial VM placement. Finally, the testbed was used to both demonstrate and conduct experimental evaluation to showcase its performance under different management objectives.

The thesis objective was met by the following deliverables:

1. A single configurable scheduler implementation, capable of handling different management objectives. The scheduler was designed and implemented to take advantage of extended host utilization metrics to improve on scheduling in an IaaS cloud. Implementations for two management objectives; balanced load and energy efficiency were included with the scheduler.

2. Two operational testbeds, one at KTH and one at Ericsson were delivered as part of this thesis. In addition to the OpenStack based cloud platform itself, they consisted of preconfigured VM images, a script for synthetic load generation, a monitoring setup based on Zabbix and a web based demonstrator tool used to showcase the system performance in real-time.
3. Preliminary results obtained from experimental evaluation on one of the testbeds, show the schedulers effectiveness under the two management objectives. Evaluation was done entirely based on synthetically generated cloud load. This is the one area of this thesis that probably would benefit most by complementary work. Some assumptions that were made about the cloud loads were based on educated guesses rather than on real cloud knowledge. With real traces of load from a production cloud, another set of experiments could be carried out to complement existing results.

During testing, some limitations of the scheduler were uncovered. The causes for these are mainly due to limitations in one of either the scheduler implementation or the OpenStack platform used for the implementation.

The first issue is related to how the implemented scheduler uses periodically collected host state to base its scheduling decisions on. The collection rate naturally limits the efficiency of the scheduler as it needs to have up-to-date information to make correct decisions. To simplify the implementation, state collection was done in-process in the existing nova-compute service that runs on every host. This service, like all nova services use a single thread to do all its work and because of this, too frequent collection of state would negatively impact all other duties of this service, like building and launching new VMs. In a production environment, state collection would most likely be done elsewhere, possibly by a monitoring system specifically designed to handle this task.

The other issue that was discovered relates to the fact that launching a new VM instance takes some considerable amount of time, which puts a bound to how fast the scheduler can work. Launching a new VM involves time consuming tasks necessary to launch a VM such as copying a base image, querying other services for network information etc. All these tasks introduce a rather big time-delay between when the VM is initially scheduled until it is actually getting started. If requests or VMs happen too frequently, they will queue up at the compute nodes, resulting in all kinds of problems. This can in many cases be resolved by scaling out the system by just adding more nodes, which for obvious reasons was not an option in our limited setup.

Finally, the method used to evaluate the scheduler's performance deserves a note. While the initial placement scheduler was being tested in isolation, this controller in most cases only constitutes part of a larger resource management system. With initial VM placement based on real-time utilization, the risk of breaking SLAs increases if not met by other means. The scheduler alone is highly dependent on loads being stable,

since it cannot reconsider the placements it has made to adjust for demand changes incurred by existing VMs on a host that results in the host becoming overloaded. One solution to this problem would be to run a dynamic placement controller in parallel with the initial placement controller, capable of adapting VM placements using a method like VM live-migration.

## 7.2 Experiences

This thesis provided an excellent opportunity to gain knowledge on cloud computing in general, but also on OpenStack, which is the most popular and growing cloud platform of today. It also gave me a chance to learn a new programming language, Python, which was a pleasant experience.

On the project side of things, a bit of more planning in the beginning would have helped (I guess I'm not the first to say this). For a lengthy project like this it is easy lose focus of the important pieces of the work and to invest too much time on details.

For different reasons I decided to do this thesis on my own. Having someone to throw ideas at can really help in a project like this. My advice to those who plan to do a thesis on their own is to consider this the very last option.

## 7.3 Future Work

While this thesis conducted a performance evaluation of the designed scheduler, many questions still remain to be answered. The cloud setup used as testbed consisted of just nine machines, which in terms of a cloud is a very small setup. To practically be able to test a larger setup, some form of simulation would most likely be required. Such tests could give valuable answers to issues of scaling that has not been part of this thesis.

The load generated during tests was synthetically generated based on some assumptions that might not be applicable to all clouds. Testing on real traces of load and/or complement current tests using other assumptions could result in more complete picture of how the scheduler performs in different scenarios.

Other areas of research might involve adding network related objectives that takes VM-to-VM properties such as communication patterns etc. into consideration. An area mostly left untouched in this thesis is also that of demand profiling to improve on the input being fed to the scheduler. Much of this would need knowledge of the applications being run in the cloud, and therefore such information would need to be made available to the scheduler.

## 8 Bibliography

- [1] "Amazon Elastic Compute Cloud (Amazon EC2)," [Online]. Available: <http://aws.amazon.com/ec2/>.
- [2] "OpenStack Open Source Cloud Computing Software," [Online]. Available: <http://www.openstack.org/>.
- [3] L. M. Vaquero, L. Rodero-Merino, J. Caceres and M. Lindner, "A Break in the Clouds: Towards a Cloud Definition," *ACM SIGCOMM Computer Communication Review*, vol. 39, no. 1, pp. 50-55, 2009.
- [4] I. Foster, Y. Zhao, I. Raicu and S. Lu, "Cloud Computing and Grid Computing 360-Degree Compared," in *Grid Computing Environments Workshop*, Austin, 2008.
- [5] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. H. Katz, A. Konwinski, G. Lee, D. A. Patterson, A. Rabkin, I. Stoica and M. Zaharia, "Above the Clouds: A Berkeley View of Cloud Computing," EECS University of California at Berkeley, 2009.
- [6] P. Mell and T. Grance, "The NIST Definition of Cloud Computing," National Institute of Standards and Technology / U.S. Department of Commerce, 2011.
- [7] "Salesforce.com: CRM and Cloud Computing To Grow Your Business," [Online]. Available: <http://www.salesforce.com/>.
- [8] "Google App Engine — Cloud Platform," [Online]. Available: <https://cloud.google.com/products/>.
- [9] "Google Compute Engine — Cloud Platform," [Online]. Available: <https://cloud.google.com/products/compute-engine>.
- [10] "Windows Azure: Microsoft's Cloud Platform | Cloud Hosting | Cloud Services," [Online]. Available: <http://www.windowsazure.com/>.
- [11] "Open Cloud Computing, Cloud Hosting, Cloud Storage by Rackspace," [Online]. Available: <http://www.rackspace.com/cloud/>.
- [12] "HP Cloud Services," [Online]. Available: <https://www.hpcloud.com/>.
- [13] "Citrix XenServer - Efficient Server Virtualization Software - Citrix," [Online]. Available: <http://www.citrix.com/products/xenserver/overview.html>.



- [14] "KVM," [Online]. Available: <http://www.linux-kvm.org/>.
- [15] "libvirt: The virtualization API," [Online]. Available: <http://libvirt.org/>.
- [16] "RESTful Web services: The basics," [Online]. Available: <http://www.ibm.com/developerworks/webservices/library/ws-restful/>.
- [17] "Python Programming Language – Official Website," [Online]. Available: <http://www.python.org/>.
- [18] F. Wuhib, R. Stadler and H. Lindgren, "Dynamic resource allocation with management objectives—Implementation for an OpenStack cloud," in *8th International Conference on Network and Service Management*, Las Vegas, 2012.
- [19] G. Jung, M. A. Hiltunen, K. R. Joshi, R. D. Schlichting and C. Pu, "Mistral: Dynamically Managing Power, Performance, and Adaptation Cost in Cloud Infrastructures," in *International Conference on Distributed Computing Systems*, Genova, 2010.
- [20] D. Jayasinghe, C. Pu, T. Eilam, M. Steinder, I. Whalley and E. Snible, "Improving Performance and Availability of Services Hosted on IaaS Clouds with Structural Constraint-aware Virtual Machine Placement," in *International Conference on Services Computing*, Washington DC, 2011.
- [21] H. Moens, J. Famaey, S. Latré, B. Dhoedt and F. De Turck, "Design and Evaluation of a Hierarchical Application Placement Algorithm in Large Scale Clouds," in *12th IFIP/IEEE International Symposium on Integrated Network Management*, Dublin, 2011.
- [22] A. Gulati, G. Shanmuganathan, A. Holler, C. Waldspurger, M. Ji and X. Zhu, "VMware Distributed Resource Management: Design, Implementation, and Lessons Learned," VMware, 2012.
- [23] "VMware vSphere 5.0 Configuration Maximums," [Online]. Available: <http://www.vmware.com/pdf/vsphere5/r50/vsphere-50-configuration-maximums.pdf>.
- [24] R. L. Graham, "Bounds for certain multiprocessing anomalies," *Bell System Technical Journal*, vol. 45, pp. 1563-1581, 1966.
- [25] S. Dutta and A. Verma, "Service Deactivation Aware Placement and Defragmentation in Enterprise Clouds," in *7th International Conference on Network and Service Management*, Paris, 2011.

- [26] J. Aspnes, Y. Azar, A. Fiat, S. Plotkin and O. Waarts, "On-Line Load Balancing with Applications to Machine Scheduling and Virtual Circuit Routing," in *25th Annual ACM Symposium on Theory of Computing*, San Diego, 1993.
- [27] Y. Azar and L. Epstein, "On-Line Load Balancing of Temporary Tasks on Identical Machines," 1997.
- [28] D. B. Shmoys, J. Wein and D. P. Williamson, "Scheduling parallel machines on-line," 1991.
- [29] C. Xian, Y.-H. Lu and Z. Li, "Energy-Aware Scheduling for Real-Time Multiprocessor Systems with Uncertain Task Execution Time," ACM, San Diego, 2007.
- [30] "Bin packing problem - Wikipedia, the free encyclopedia," [Online]. Available: [http://en.wikipedia.org/wiki/Bin\\_packing\\_problem](http://en.wikipedia.org/wiki/Bin_packing_problem).
- [31] "IBM OpenStack, OpenStack nova-scheduler and its algorithm," [Online]. Available: [https://www.ibm.com/developerworks/community/blogs/e93514d3-c4f0-4aa0-8844-497f370090f5/entry/openstack\\_nova\\_scheduler\\_and\\_its\\_algorithm27](https://www.ibm.com/developerworks/community/blogs/e93514d3-c4f0-4aa0-8844-497f370090f5/entry/openstack_nova_scheduler_and_its_algorithm27).
- [32] "PHP: Hypertext Preprocessor," [Online]. Available: <http://php.net/>.
- [33] "Flot: Attractive JavaScript plotting for jQuery," [Online]. Available: <http://www.flotcharts.org/>.
- [34] "Ajax (programming) - Wikipedia, the free encyclopedia," [Online]. Available: [http://en.wikipedia.org/wiki/Ajax\\_\(programming\)](http://en.wikipedia.org/wiki/Ajax_(programming)).
- [35] "Joomla! The CMS Trusted By Millions for their Websites," [Online]. Available: <http://www.joomla.org/>.
- [36] "VMware vSphere Private Cloud Computing & Virtualization," [Online]. Available: <http://www.vmware.com/products/datacenter-virtualization/vsphere/overview.html>.